МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ

ДЕРЖАВНИЙ УНІВЕРСИТЕТ «ЖИТОМИРСЬКА ПОЛІТЕХНІКА»

Кафедра інженерії програмного забезпечення

КУРСОВА РОБОТА

з дисципліни «Моделювання та аналіз програмного забезпечення»

на тему:

«Програмна платформа автоматизованого керування житловим середовищем»

студента 4 курсу групи ІПЗ-22-2
спеціальності 121 «Інженерія
програмного забезпечення»
Стецюка Богдана Юрійовича
(прізвище, ім'я та по-батькові)

Керівник: доцент кафедри КН
Левківський В.Л.

Дата захисту: "___" грудня 2025р.
Національна шкала _____
Кількість балів: _____
Оцінка: ECTS _____

Члени комісії _____ Сугоняк І.І.
(підпис) (прізвище та ініціали)
_____ Кравченко С.М.
(підпис) (прізвище та ініціали)
_____ Левківський В.Л.
(підпис) (прізвище та ініціали)

Житомир – 2025

ДЕРЖАВНИЙ УНІВЕРСИТЕТ «ЖИТОМИРСЬКА ПОЛІТЕХНІКА»
Факультет інформаційно-комп'ютерних технологій
Кафедра інженерії програмного забезпечення
Освітній рівень: бакалавр
Спеціальність 121 «Інженерія програмного забезпечення»

«ЗАТВЕРДЖУЮ»
Завідувач кафедри ІПЗ
_____ Тетяна Вакалюк
"____"_____2025 р.

ЗАВДАННЯ
НА КУРСОВУ РОБОТУ СТУДЕНТУ
Стецюку Богдану Юрійовичу

1. Тема роботи: Програмна платформа автоматизованого керування житловим середовищем,
   керівник роботи: доцент кафедри КН Левківський В.Л.
2. Строк подання студентом: "___"_грудня_ 2025 р.
3. Вихідні дані до роботи: Розробити архітектуру для системи автоматизованого керування житловим середовищем.
4. Зміст розрахунково-пояснювальної записки (перелік питань, які підлягають розробці):
   1. Постановка завдання
   2. Аналіз вимог користувача
   3. Модель програмного комплексу на логічному рівні
   4. Фізична модель
   5. Прототип програмного комплексу

5. Перелік графічного матеріалу (з точним зазначенням обов'язкових креслень):
   1. Презентація
   2. Діаграми

6. Консультанти розділів проекту (роботи)

Розділ	Прізвище, ініціали та посади консультанта	Підпис, дата	
		завдання видав	завдання прийняв

7. Дата видачі завдання “___” _____ 2025 р.

КАЛЕНДАРНИЙ ПЛАН

№ з/п	Назва етапів курсової роботи	Строк виконання етапів проекту	Примітки
1	Постановка задачі	10.09.2025	Виконано
2	Аналіз вимог користувача	12.09.2025	Виконано
3	Формулювання технічного завдання	15.10.2025	Виконано
4	Опрацювання літературних джерел	16.10.2025	Виконано
5	Моделювання системи на логічному рівні	27.10.2025	Виконано
6	Фізична модель	03.11.2025	Виконано
7	Генерування програмного коду для прототипу	10.12.2025	Виконано
8	Написання пояснювальної записки	19.12.2025	Виконано
9	Захист	26.12.2025	Виконано

Студент _____ Стецюк Б.Ю.
(підпис) (прізвище та ініціали)

Керівник проекту _____ Левківський В. Л.
(підпис) (прізвище та ініціали)

РЕФЕРАТ

Пояснювальна записка до курсової роботи на тему «Програмна платформа автоматизованого керування житловим середовищем» складається зі вступу, трьох розділів, висновків, списку використаних джерел та додатків.

Текстова частина викладена на 46 сторінках друкованого тексту. Джерела інформації містять 14 найменувань і займають 2 сторінки. Пояснювальна записка має 10 сторінок додатків. У роботі наведено 15 рисунків. Загальний обсяг роботи – 58 сторінок.

У першому розділі виконано аналіз вимог до програмної системи, визначено основні функціональні та нефункціональні вимоги, описано варіанти використання та ролі користувачів.

У другому розділі розроблено логічну модель програмного комплексу, що включає діаграми активностей, діаграму класів та діаграми послідовності, які відображають взаємодію компонентів системи та алгоритми її роботи.

У третьому розділі представлено фізичну модель системи, описано архітектуру програмного комплексу та взаємодію його компонентів, а також наведено діаграму компонентів, що відображає структуру системи.

У висновках узагальнено результати виконаної роботи, зокрема аналіз вимог, побудову моделей та дослідження архітектури програмної платформи.

Метою роботи є моделювання та аналіз програмної платформи автоматизованого керування житловим середовищем із використанням UML та CASE-засобів.

У додатках розміщено каталог вимог та глосарій.

ЗМІСТ

РЕФЕРАТ 4
ВСТУП 6
РОЗДІЛ 1. АНАЛІЗ ВИМОГ КОРИСТУВАЧА ТА КОНЦЕПТУАЛЬНЕ МОДЕЛЮВАННЯ 7
1.1. Технічне завдання на розробку системи 7
1.2. Обґрунтування вибору засобів моделювання 13
1.3. Аналіз вимог до програмного продукту 15
Висновки до першого розділу 18
РОЗДІЛ 2. РОЗРОБКА МОДЕЛІ ПРОГРАМНОГО КОМПЛЕКСУ НА ЛОГІЧНОМУ РІВНІ 19
2.1. Алгоритм роботи та стани програмної системи 19
2.2. Об’єктно-орієнтована модель системи 27
2.3. Комунікації і послідовність взаємодії об’єктів системи 34
Висновки до другого розділу 38
РОЗДІЛ 3. ФІЗИЧНА МОДЕЛЬ ТА ПРОТОТИП ПРОГРАМНОГО КОМПЛЕКСУ 39
3.1. Взаємодія компонентів системи 39
3.2. Архітектура програмного комплексу та його розгортання 40
3.3. Генерування програмного коду для прототипу програмного комплексу 42
Висновки до третього розділу 43
ВИСНОВКИ 46
ДЖЕРЕЛА ІНФОРМАЦІЇ 47
ДОДАТКИ 49

ВСТУП

У сучасних умовах стрімкого розвитку інформаційних технологій особливу увагу приділяють автоматизації повсякденних процесів. Одним із перспективних напрямків є створення програмних платформ для автоматизованого керування житловим середовищем, які забезпечують підвищення комфорту, безпеки та ефективності використання ресурсів.

Концепція «розумного будинку» (Smart Home) передбачає інтеграцію різноманітних пристроїв, сенсорів та систем керування у єдину інформаційну платформу. Це дозволяє користувачам здійснювати централізоване керування освітленням, опаленням, системами безпеки та іншими компонентами житлового середовища як у ручному, так і в автоматизованому режимі.

Актуальність даної теми обумовлена широким впровадженням технологій Інтернету речей (IoT), що забезпечують взаємодію між пристроями, а також зростанням попиту на інтелектуальні системи керування житлом. Використання таких систем дозволяє оптимізувати споживання енергії, підвищити рівень безпеки та забезпечити зручність керування житловим середовищем.

Метою курсової роботи є моделювання програмної платформи автоматизованого керування житловим середовищем із використанням UML та CASE-засобів.

Для досягнення поставленої мети необхідно вирішити такі задачі:

- провести аналіз вимог до програмної системи;
- визначити варіанти використання системи;
- побудувати UML-діаграми активностей, класів та послідовності;
- розробити логічну модель програмного комплексу;
- змоделювати архітектуру системи та взаємодію її компонентів.

Об’єктом дослідження є процеси автоматизованого керування житловим середовищем.

Предметом дослідження є методи та засоби моделювання програмного забезпечення з використанням UML та CASE-технологій.

РОЗДІЛ 1. АНАЛІЗ ВИМОГ КОРИСТУВАЧА ТА КОНЦЕПТУАЛЬНЕ МОДЕЛЮВАННЯ

1.1. Технічне завдання на розробку системи

Найменування програмного засобу

Повне найменування програмної системи: «Програмна платформа автоматизованого керування житловим середовищем» (надалі — система).

Коротка назва програмної системи — «Smart Home Platform».

Призначення розробки та область застосування

Програмна система призначена для автоматизації процесів керування житловим середовищем шляхом інтеграції різноманітних пристроїв, сенсорів та керуючих модулів у єдину інформаційну платформу.

Система може застосовуватись:

- у приватних будинках і квартирах;
- у комерційних приміщеннях;
- у системах «розумного будинку».

Мета

Метою розробки є створення програмної платформи, яка забезпечує централізоване керування житловим середовищем, моніторинг стану пристроїв та автоматизацію процесів за допомогою сценаріїв.

Найменування розробника та замовника

Розробник — студент групи ІПЗ-22-2.

Замовник — кафедра інженерії програмного забезпечення Державного університету «Житомирська політехніка».

Документ на підставі якого ведеться розробка

Розробка виконується відповідно до навчального плану за спеціальністю 121 «Інженерія програмного забезпечення» в межах дисципліни «Моделювання та аналіз програмного забезпечення».

Вимоги до функціональних характеристик

Програмна система повинна забезпечувати:

- реєстрацію та автентифікацію користувачів;

- управління ролями користувачів (звичайний користувач, адміністратор);
- додавання, редагування та видалення пристроїв;
- перегляд списку пристроїв із можливістю фільтрації;
- керування пристроями (увімкнення, вимкнення, зміна режимів роботи);
- моніторинг стану пристроїв у реальному часі;
- створення, редагування та виконання сценаріїв автоматизації;
- отримання сповіщень про події та зміни стану;
- ведення журналу подій системи;
- централізоване керування доступом користувачів.

Організація вхідних і вихідних даних

Вхідні дані:

- облікові дані користувачів;
- параметри та налаштування пристроїв;
- дані сенсорів (температура, вологість, рух тощо);
- налаштування сценаріїв автоматизації.

Вихідні дані:

- інформація про стан пристроїв;
- сповіщення користувачів;
- результати виконання сценаріїв;
- записи журналу подій.

Часові характеристики і розмір пам'яті, необхідної для роботи програми

Часові характеристики системи визначають допустимі межі затримок при виконанні основних операцій та забезпечують комфортну взаємодію користувача з програмною платформою.

Система повинна відповідати наступним вимогам:

- Час відгуку інтерфейсу користувача при виконанні стандартних операцій (перегляд списку пристроїв, відкриття сторінок, зміна стану пристрою) не повинен перевищувати 1 секунди.
- Час виконання операцій керування пристроями (надсилання команди, обробка та отримання відповіді) не повинен перевищувати 1–2 секунд, враховуючи затримки мережі та обробку на стороні контролера.
- Час обробки сценаріїв автоматизації залежить від складності сценарію, але не повинен перевищувати 3–5 секунд з моменту спрацювання тригера до завершення виконання всіх дій.
- Час отримання та відображення даних від сенсорів (температура, вологість, рух) повинен забезпечуватись у режимі, близькому до реального часу, із затримкою не більше 1 секунди.
- Час авторизації користувача не повинен перевищувати 2 секунд, включаючи перевірку облікових даних та встановлення сесії.
- Пропускна здатність системи повинна забезпечувати одночасну роботу не менше 500 активних користувачів без суттєвого зниження продуктивності.
- Масштабованість системи повинна дозволяти збільшення кількості користувачів до 1000 і більше шляхом горизонтального або вертикального масштабування без значної деградації часу відгуку.
- Час відновлення після збою (перезапуск сервісів, відновлення з’єднання) не повинен перевищувати 1–2 хвилин.

Вимоги до надійного функціонування

Програмна система повинна забезпечувати стабільну та безперебійну роботу за умови коректного функціонування апаратного забезпечення та наявності доступу до мережі Інтернет. У разі виникнення апаратних або програмних збоїв система повинна забезпечувати відновлення працездатності та збереження даних.

У разі апаратних збоїв:

До апаратних збоїв належать відмови серверного обладнання, клієнтських пристроїв або мережевої інфраструктури.

Система повинна забезпечувати:

- відновлення роботи після перезапуску серверного або клієнтського обладнання;
- збереження даних у базі даних за допомогою механізмів резервного копіювання;
- автоматичне повторне підключення до серверів після відновлення мережі;
- мінімізацію втрати даних за рахунок використання транзакцій та журналювання;
- можливість використання резервних серверів або механізмів масштабування (за наявності).

У разі програмних збоїв:

До програмних збоїв належать помилки у роботі програмного забезпечення, некоректна обробка запитів або виняткові ситуації.

Система повинна забезпечувати:

- обробку виняткових ситуацій без аварійного завершення роботи системи;
- ведення журналу помилок для подальшого аналізу та усунення;
- можливість повторного виконання операцій у разі їх некоректного завершення;
- контроль цілісності даних при виконанні операцій;
- ізоляцію помилок окремих компонентів без впливу на всю систему.

Додаткові вимоги:

Для забезпечення високого рівня надійності система повинна відповідати таким вимогам:

- Автоматичне збереження стану системи — всі критичні дані повинні зберігатися у режимі реального часу;

- Журналювання подій (логування) — фіксація всіх ключових дій користувачів і системних подій;
- Моніторинг стану системи — можливість відстеження стану компонентів та виявлення збоїв;
- Резервне копіювання — регулярне створення резервних копій бази даних;
- Відновлення після збоїв — можливість швидкого повернення системи до робочого стану;
- Масштабованість — система повинна підтримувати збільшення навантаження без втрати стабільності.

Контроль вхідної і вихідної інформації

Для забезпечення коректної роботи програмної платформи та запобігання помилкам користувача система повинна здійснювати перевірку вхідних і вихідних даних.

Перевірка відповідності даних

Система повинна виконувати валідацію введених даних, зокрема:

- перевірку коректності формату електронної пошти, пароля та обов'язкових полів під час реєстрації;
- перевірку числових значень параметрів пристроїв (наприклад, температура, рівень освітлення);
- контроль допустимих значень для налаштувань сценаріїв автоматизації;
- перевірку полів із фіксованими значеннями (тип пристрою, статус, роль користувача).

Автозаповнення та вибір із переліку

Для зменшення кількості помилок введення система повинна використовувати:

- випадаючі списки для вибору типів пристроїв, кімнат або сценаріїв;
- автозаповнення при введенні даних (наприклад, пошук пристроїв або приміщень);

- попередньо визначені значення для типових налаштувань.

Додаткові заходи

Для підвищення надійності та безпеки система повинна забезпечувати:

- обмеження розміру та формату даних, що передаються (наприклад, параметри пристроїв або конфігурації);
- використання обов'язкових полів для ключових налаштувань системи;
- встановлення таймаутів для запитів з метою уникнення дублювання операцій;
- захист від типових вразливостей, таких як SQL-ін'єкції, XSS-атаки та інші шкідливі запити;
- перевірку коректності даних, що надходять від зовнішніх пристроїв та сенсорів.

Час відновлення після відмови

Час відновлення роботи програмної платформи після відмови включає перезапуск сервера, повторне підключення до бази даних, автоматичне відновлення останніх збережених даних, а також обробку незавершених операцій, таких як виконання сценаріїв або передача команд керування пристроями. У разі стандартних збоїв система повинна відновлюватися протягом 1-2 хвилин, а у випадку серйозних відмов — не більше ніж за 1 годину. Для мінімізації простою передбачається використання резервних серверів та механізмів відмовостійкості.

Умови експлуатації і збереження

Програмна платформа функціонує в багатокористувацькому середовищі з постійним доступом до мережі Інтернет. Збереження даних забезпечується шляхом регулярного автоматичного створення резервних копій. Резервні копії зберігаються на захищених серверах і використовуються для відновлення системи у разі збоїв або втрати даних. Такий підхід забезпечує надійність роботи системи та мінімізує ризик втрати критичної інформації.

Вимоги до інформаційної і програмної сумісності.

Програмна система повинна забезпечувати коректну роботу у сучасних веб-браузерах, таких як Google Chrome, Mozilla Firefox, Microsoft Edge, Safari. Інтерфейс користувача та функціональні можливості системи повинні однаково відображатися незалежно від обраного браузера та його версії, за умови підтримки сучасних веб-технологій.

Система повинна підтримувати адаптивний інтерфейс для коректної роботи на різних типах пристроїв, включаючи персональні комп'ютери, планшети та смартфони.

Вимоги до складу і параметрів технічних засобів

Система повинна забезпечувати стабільну роботу на сервері та клієнтських пристроях із використанням сучасних технологій веб-розробки.

Для реалізації програмної платформи можуть використовуватись наступні технології:

- React.js (v18.2.0) для розробки користувацького інтерфейсу;
- Node.js (v18 або вище) як серверне середовище;
- PostgreSQL або інша база даних для зберігання інформації про користувачів, пристрої, сценарії та події системи.

Серверна інфраструктура повинна забезпечувати:

- стабільну роботу системи при одночасному доступі користувачів;
- обробку запитів у режимі реального часу;
- можливість масштабування при зростанні навантаження;
- взаємодію з зовнішніми пристроями та IoT-контролерами.

Вимоги до програмної документації

Програмна документація повинна включати наступні відомості:

1. Документацію API, що описує структуру запитів і відповідей, доступні ендпоінти, HTTP-методи (GET, POST, PUT, DELETE) та приклади використання для інтеграції з іншими компонентами системи або зовнішніми сервісами.

2. Користувацьку документацію, яка містить опис основних функцій системи, інструкції щодо керування пристроями, створення сценаріїв та

роботи з інтерфейсом. У межах курсової роботи відповідні відомості відображені в розділах, присвячених аналізу вимог та моделюванню системи.

Обґрунтування вибору засобів моделювання

У процесі вибору інструменту для створення UML-діаграм було проведено аналіз декількох популярних рішень, зокрема Visual Paradigm, Draw.io, Lucidchart та StarUML.

Visual Paradigm

Переваги:

- широкий набір функцій для створення UML-діаграм та інших типів моделей (BPMN, ERD, SysML);
- наявність інструментів для командної роботи та керування проєктами.

Недоліки:

- висока вартість повної версії;
- складність використання окремих функцій.

Draw.io

Переваги:

- безкоштовний доступ і можливість роботи через браузер без встановлення;
- підтримка різних платформ (Windows, MacOS, Linux, Android, iOS);
- широкий вибір форматів експорту (PNG, JPEG, SVG, PDF);
- наявність готових шаблонів та підтримка хмарного зберігання.

Недоліки:

- відсутність можливості генерації коду;
- обмежена функціональність для складних проєктів.

Lucidchart

Переваги:

- інтеграція з популярними сервісами (Slack, AWS, Google Workspace тощо);

- можливість роботи з даними в реальному часі.

Недоліки:

- обмеження функціоналу у безкоштовній версії;
- залежність від веб-інтерфейсу.

StarUML

Переваги:

- зручний інтерфейс для створення UML-діаграм;
- підтримка генерації коду;
- наявність плагінів для розширення функціональності.

Недоліки:

- обмежені можливості налаштування елементів;
- відсутність розширених інструментів для точного розміщення компонентів.

Висновок щодо вибору інструменту

Для моделювання програмної платформи автоматизованого керування житловим середовищем було обрано мову UML та інструмент Draw.io.

Обраний інструмент забезпечує достатній функціонал для створення необхідних діаграм (варіантів використання, активностей, класів, послідовності та компонентів), є простим у використанні та не потребує додаткових витрат. Це дозволяє ефективно реалізувати процес моделювання системи в межах курсової роботи.

1.2. Аналіз вимог до програмного продукту

Для забезпечення ефективного функціонування програмної платформи автоматизованого керування житловим середовищем були визначені основні вимоги, яким повинна відповідати система. Ці вимоги спрямовані на забезпечення зручності використання, надійності та безпеки роботи платформи для всіх користувачів. Каталог вимог і глосарій термінів наведено в додатках А та Б відповідно.

Аналіз вимог користувачів дозволив визначити варіанти використання системи. На рисунку 1.1 наведено діаграму варіантів використання програмної платформи.

У системі автоматизованого керування житловим середовищем розрізняють два основних актори: користувач (user) та адміністратор (admin).

Короткий опис акторів представлено в таблиці 1.1.

Таблиця 1.1

Виявлення акторів

Актор	**Короткий опис**
Адміністратор (Admin)	Здійснює управління користувачами системи, контролює налаштування платформи, пристроїв та сценаріїв. Має доступ до всіх функціональних можливостей системи.
Користувач (User)	Має можливість керувати пристроями житлового середовища, переглядати їх стан, створювати та використовувати сценарії автоматизації, а також отримувати сповіщення про події в системі.

Таблиця 1.2

Виявлення варіантів використання

Основний актор	**Найменування**	**Формулювання**
Користувач/ Адміністратор	Авторизація	Цей варіант дозволяє вводити облікові дані та отримувати доступ до системи
	Реєстрація	Цей варіант дозволяє створювати обліковий запис у системі
	Перегляд особистих даних	Цей варіант дозволяє переглядати персональну інформацію користувача
	Зміна особистих даних	Цей варіант дозволяє змінювати особисті дані
	Перегляд пристроїв	Цей варіант дозволяє переглядати список підключених пристроїв та їх стан

	Редагування/видалення пристроїв	Цей варіант дозволяє змінювати або видаляти налаштовані пристрої
Користувач	Керування пристроями	Цей варіант дозволяє змінювати стан пристроїв (увімкнення, вимкнення, зміна режиму)
	Створення сценаріїв автоматизації	Цей варіант дозволяє створювати сценарії для автоматичного керування житловим середовищем
	Запуск сценаріїв	Цей варіант дозволяє виконувати створені сценарії автоматизації
	Отримання сповіщень	Цей варіант дозволяє отримувати повідомлення про події та зміни стану системи
	Перегляд журналу подій	Цей варіант дозволяє переглядати історію подій та дій у системі
Адміністратор	Управління користувачами платформи	Цей варіант дозволяє адміністратору створювати, редагувати, блокувати користувачів та змінювати їх ролі
	Управління пристроями та сценаріями	Цей варіант дозволяє контролювати налаштування пристроїв і сценаріїв системи
	Перегляд аналітики системи	Цей варіант дозволяє переглядати загальну інформацію про стан системи, активність користувачів та події

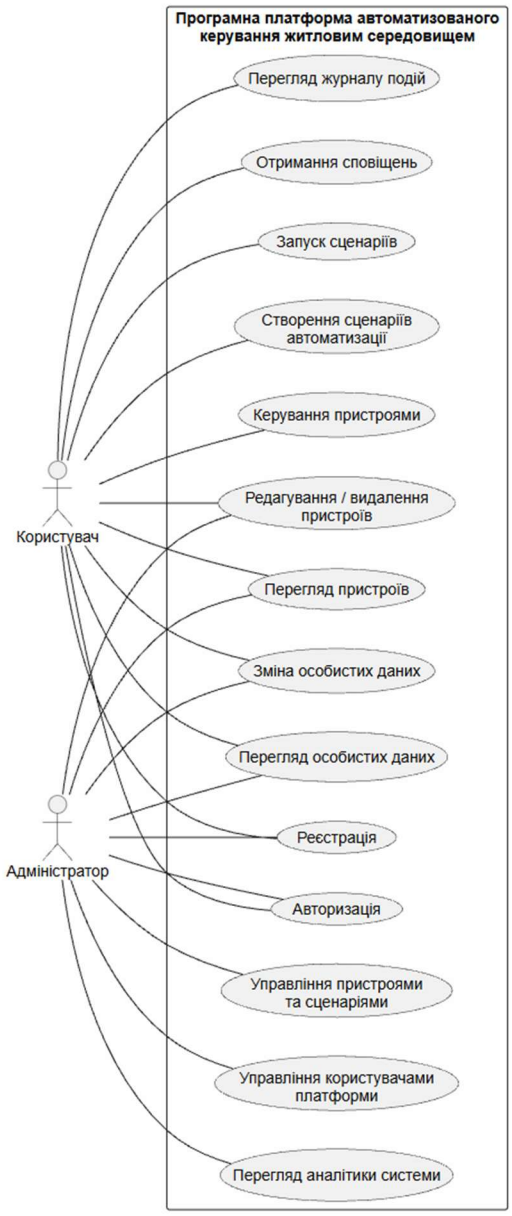

Рис. 1.1. Діаграма варіантів використання системи

Висновки до першого розділу

У першому розділі було проаналізовано вимоги до програмної платформи автоматизованого керування житловим середовищем та визначено її основні функціональні можливості. Також було виділено ключових акторів системи та сформовано варіанти її використання, що дозволило описати взаємодію користувачів із системою. Отримані результати є основою для подальшого моделювання системи.

РОЗДІЛ 2. РОЗРОБКА МОДЕЛІ ПРОГРАМНОГО КОМПЛЕКСУ НА ЛОГІЧНОМУ РІВНІ

2.1. Алгоритм роботи та стани програмної системи

Алгоритми роботи та стани програмної системи є важливими складовими її архітектури, які визначають логіку виконання функцій та взаємодію між компонентами. Чітке визначення цих аспектів забезпечує коректну роботу системи, спрощує її розробку та подальше вдосконалення.

Алгоритми описують послідовність дій, які виконує система для досягнення певної мети. У контексті програмної платформи автоматизованого керування житловим середовищем це можуть бути як прості процеси (наприклад, авторизація користувача), так і складні сценарії, пов’язані з керуванням пристроями або виконанням автоматизації.

Стан системи відображає її поточний режим роботи, наприклад: очікування дій користувача, обробка запиту, виконання команди або завершення операції.

Для моделювання алгоритмів роботи системи використовуються діаграми активності. Вони дозволяють наочно представити:

- послідовність дій під час виконання операцій;
- розгалуження логіки залежно від умов;
- взаємодію користувача та системи;
- можливість паралельного виконання процесів.

Діаграми активності використовуються для візуалізації основних процесів роботи системи, зокрема:

- авторизації користувача;
- керування пристроями;
- створення та виконання сценаріїв автоматизації.

Зокрема, процес авторизації є одним із базових, оскільки забезпечує доступ користувача до функціональних можливостей системи. Під час авторизації система перевіряє коректність введених даних та надає доступ відповідно до ролі користувача. Діаграма активності на рисунку 2.1 відображає процес авторизації користувача.

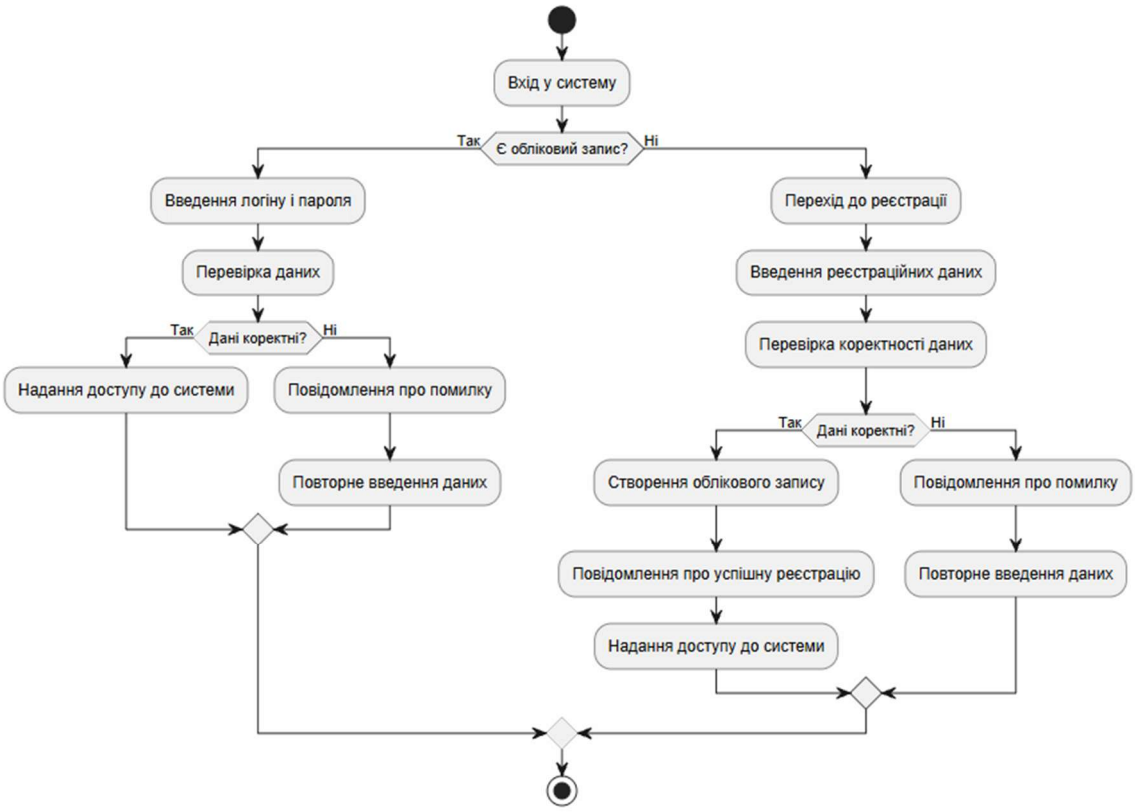

Рис. 2.1. Діаграма активності процесу авторизації та реєстрації користувача

Діаграма активності описує процес входу користувача в систему або створення нового облікового запису в межах програмної платформи автоматизованого керування житловим середовищем. Процес складається з таких етапів:

1. **Вхід у систему:**

- Користувач ініціює процес входу до системи.

2. Перевірка наявності облікового запису:

- система визначає, чи існує обліковий запис користувача;

- якщо обліковий запис існує, процес переходить до етапу автентифікації;
- якщо обліковий запис відсутній, користувачу пропонується створити новий.

3. Автентифікація користувача:

- користувач вводить облікові дані;
- система перевіряє їх коректність;
- у разі успішної перевірки користувач отримує доступ до системи;
- у випадку помилки система повідомляє користувача та пропонує повторити введення даних.

4. Створення нового облікового запису:

- користувач вводить реєстраційні дані;
- система перевіряє правильність введеної інформації;
- після успішної реєстрації створюється новий обліковий запис;
- користувач отримує повідомлення про успішне створення облікового запису.

5. Надання доступу до системи:

- після успішної автентифікації або реєстрації система надає користувачу доступ до функціональних можливостей платформи.

6. Завершення процесу:

- після виконання всіх операцій процес входу або реєстрації завершується.

Діаграма активності, наведена на рисунку 2.2, відображає процес керування пристроєм у межах програмної платформи автоматизованого керування житловим середовищем.

Процес починається з відкриття користувачем списку доступних пристроїв, вибору потрібного пристрою та задання команди керування. Після цього платформа перевіряє права доступу користувача та доступність вибраного пристрою.

Якщо пристрій доступний, система формує команду керування та передає її контролеру. Контролер виконує команду, надсилає результат до платформи, після чого система оновлює стан пристрою, фіксує подію в журналі та відображає результат користувачу.

У випадку, якщо пристрій недоступний, система повідомляє користувача про помилку без виконання команди.

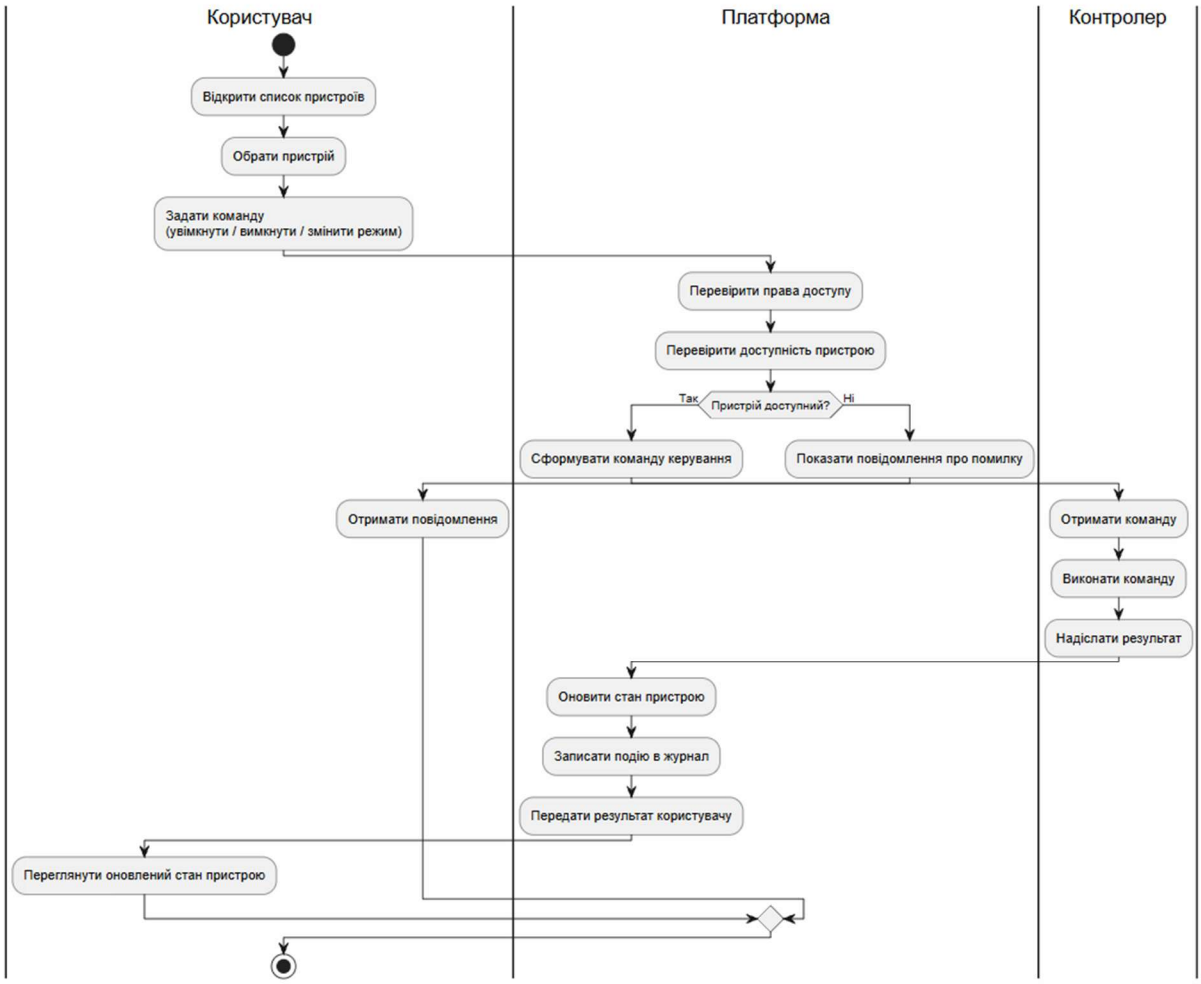

Рис. 2.2. Діаграма активності процесу керування пристроєм

Діаграма активності, наведена на рисунку 2.3, відображає процес виконання сценарію автоматизації в межах програмної платформи автоматизованого керування житловим середовищем.

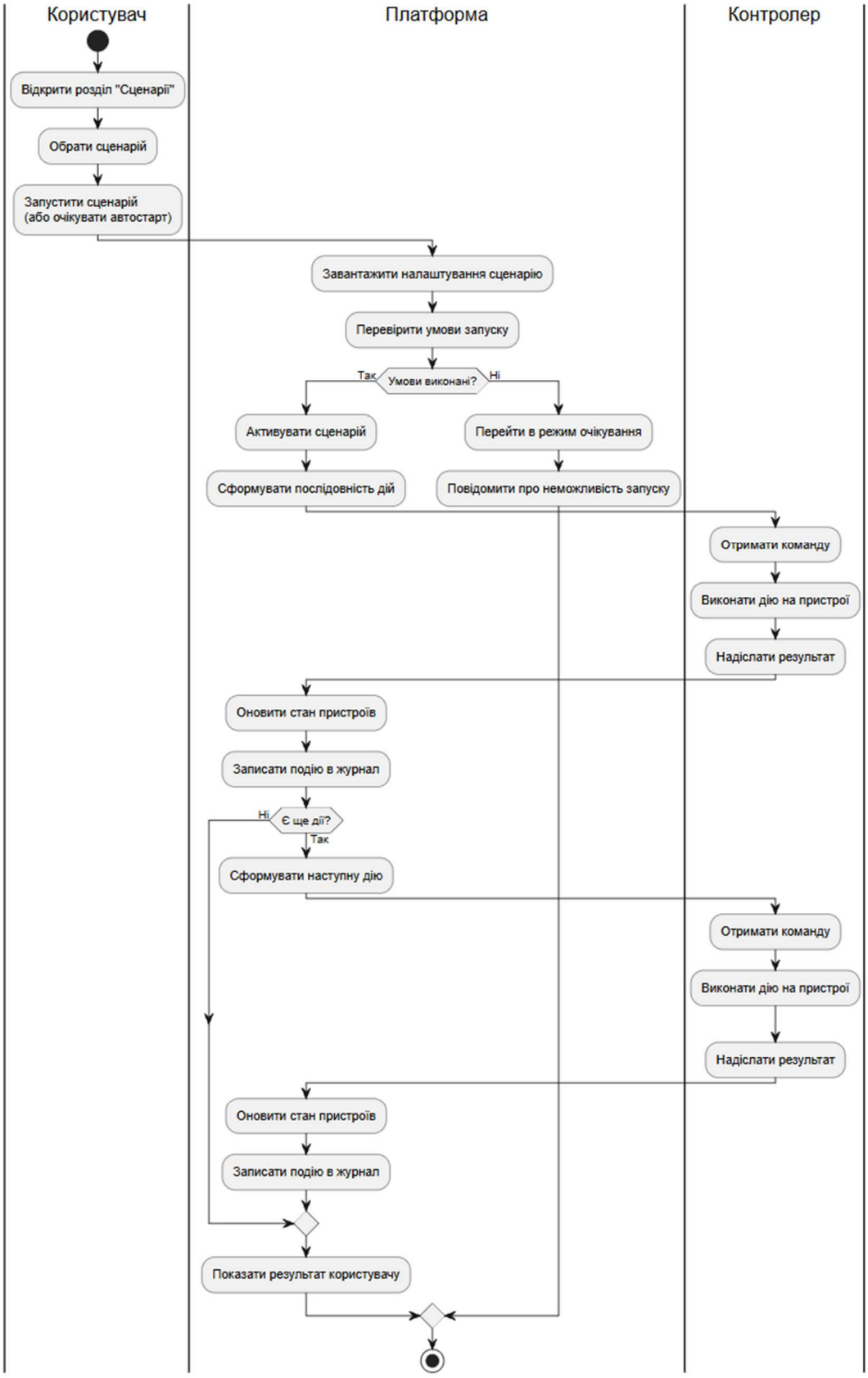

Рис. 2.3. Діаграма активності процесу виконання сценарію автоматизації

Процес включає взаємодію між користувачем і системою. Користувач може ініціювати запуск сценарію вручну або система може активувати його автоматично відповідно до заданого тригера.

Після запуску сценарію система перевіряє умови його виконання. Якщо умови не виконані, система переходить у стан очікування. У випадку виконання умов сценарій активується, після чого система послідовно виконує набір дій, пов'язаних із керуванням пристроями.

Кожна дія передається до відповідного контролера, який виконує команду на пристрої та повертає результат виконання. У разі успішного виконання система оновлює стан пристроїв та фіксує подію у журналі.

Після завершення всіх дій сценарію система оновлює інформацію для користувача та завершує процес виконання.

Таким чином, діаграма відображає логіку роботи сценаріїв автоматизації, включаючи перевірку умов, виконання дій та обробку результатів.

Опис процесів

1. Користувач

- Запуск сценарію: користувач має можливість вручну запустити заздалегідь налаштований сценарій автоматизації.
- Очікування автоматичного запуску: якщо сценарій налаштований на автоматичне виконання, користувач може лише переглядати його стан та результат виконання.

2. Система

- Ініціація виконання сценарію: система отримує запит на запуск сценарію або активує його автоматично відповідно до заданого тригера.
- Перевірка умов запуску: система перевіряє, чи виконані умови, необхідні для запуску сценарію.
    - Якщо умови не виконані, сценарій не запускається, а система переходить у режим очікування.

- Якщо умови виконані, система активує сценарій і починає послідовне виконання заданих дій.

- Виконання дій сценарію: система формує команди для відповідних пристроїв і передає їх через контролер.
- Оновлення інформації: після виконання кожної дії система оновлює стан пристроїв, зберігає подію в журналі та відображає актуальну інформацію користувачу.

3. Завершення виконання сценарію

- Після виконання всіх дій сценарію система оновлює інформацію про стан житлового середовища та завершує процес.
- У разі неможливості виконання окремої дії система фіксує помилку та повідомляє користувача.

Діаграма станів на рисунку 2.4 відображає основні етапи виконання сценарію автоматизації, включаючи такі стани, як «Очікування запуску», «Перевірка умов», «Виконання сценарію», «Оновлення стану» та «Завершення процесу». На відміну від діаграми активності, вона фокусується саме на зміні станів системи без деталізації виконавців окремих дій. Ця діаграма дозволяє зосередитися на ключових переходах між станами системи під час виконання сценарію автоматизації та відображає загальну логіку її роботи.

Рис. 2.4. Діаграма станів процесу виконання сценарію автоматизації

2.2. Об’єктно-орієнтована модель системи

Визначивши ключові ролі системи, варіанти використання, поведінкову модель та основні процеси, можна перейти до побудови об’єктно-орієнтованої моделі. На цьому етапі визначаються основні сутності програмної платформи, їх атрибути, методи та зв’язки між ними, що дозволяє формалізувати структуру системи та її внутрішню логіку.

Об’єктно-орієнтована модель дає змогу відобразити архітектуру програмної платформи автоматизованого керування житловим середовищем у вигляді діаграми класів. Така діаграма є основою для подальшої реалізації системи, оскільки визначає ключові об’єкти, що беруть участь у роботі платформи, та способи їх взаємодії.

Програмна платформа автоматизованого керування житловим середовищем включає такі основні сутності:

User — представляє користувача платформи, зберігає його облікові дані, роль та інформацію профілю;

Home — описує житловий об’єкт, у межах якого об’єднуються кімнати, пристрої, сценарії та події;

Room — представляє окреме приміщення або зону житлового об’єкта;

Controller — відповідає за взаємодію платформи з фізичними пристроями;

Device — описує керований пристрій житлового середовища;

Sensor — зберігає інформацію про сенсори та значення, які вони фіксують;

Scenario — представляє сценарій автоматизації;

Trigger — визначає умову запуску сценарію;

Action — описує конкретну дію, яка виконується в межах сценарію;

Notification — забезпечує інформування користувача про події системи;

EventLog — використовується для ведення журналу подій.

Зв’язки між сутностями системи визначають структуру їх взаємодії та рівень залежності між об’єктами. Основними типами зв’язків у моделі є асоціація та композиція.

Між класами User та Home використовується асоціація, оскільки користувач може мати доступ до одного або кількох житлових об’єктів. При цьому житловий об’єкт не залежить повністю від одного користувача та може бути пов’язаний із кількома користувачами.

Між класами Home та Room використовується композиція, оскільки кімнати існують лише в межах конкретного житлового об’єкта. Видалення об’єкта Home призводить до видалення всіх пов’язаних із ним кімнат.

Між класами Room та Device використовується асоціація, оскільки пристрої логічно прив’язуються до певної кімнати, що дозволяє організувати керування ними за розташуванням.

Зв’язок між Home та Controller є асоціацією, оскільки один житловий об’єкт може містити один або кілька контролерів, через які відбувається керування пристроями.

Між класами Controller та Device використовується асоціація, оскільки контролер передає команди пристроям і отримує від них інформацію про стан.

Між класами Scenario та Trigger, а також Scenario та Action використовується композиція, оскільки тригер і набір дій є складовими конкретного сценарію. Видалення сценарію призводить до видалення пов’язаних із ним тригера та дій.

Між класами User та Notification використовується асоціація, оскільки користувач отримує сповіщення про події, зміни стану пристроїв або результати виконання сценаріїв.

Між класами Home та EventLog використовується асоціація, оскільки журнал подій ведеться в межах конкретного житлового об’єкта та містить інформацію про виконані дії, помилки та зміни стану системи.

Таким чином, побудована об’єктно-орієнтована модель дозволяє відобразити основні сутності програмної платформи автоматизованого

керування житловим середовищем, їх властивості та взаємозв’язки, що є основою для подальшої реалізації системи.

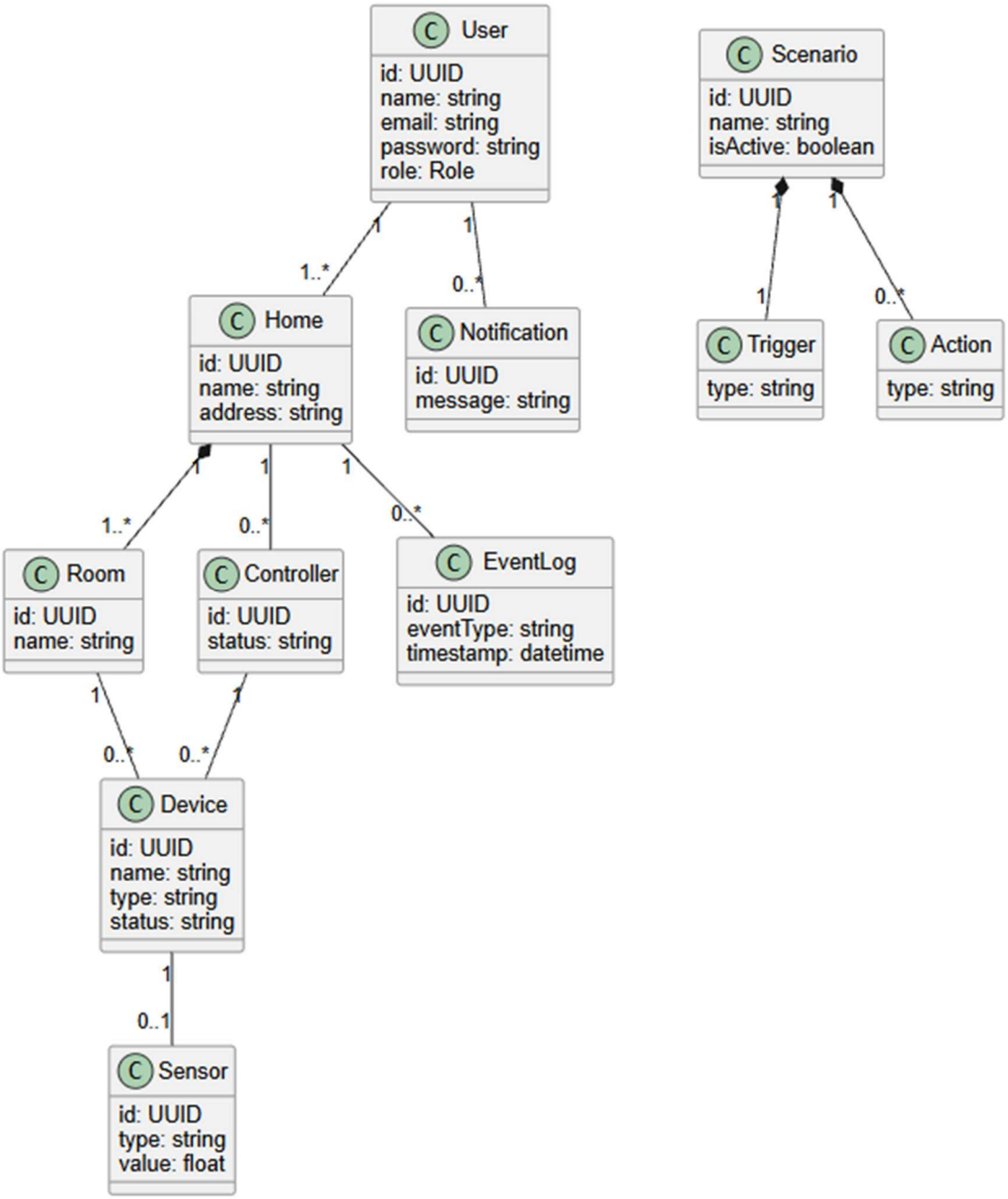

Рис. 2.5. Діаграма класів без використання патернів

Патерн «Спостерігач» (Observer) є одним із базових поведінкових патернів програмування. Його основна ідея полягає у створенні механізму, що дозволяє об'єкту (суб'єкту) сповіщати інші об'єкти (спостерігачі) про свої зміни без жорсткої залежності між ними. Це забезпечує гнучкість системи та спрощує її масштабування.

У програмній платформі автоматизованого керування житловим середовищем патерн «Спостерігач» реалізується через взаємодію між класами Device, Scenario та User. У цьому контексті пристрої або сценарії виступають як суб'єкти, які генерують події (зміна стану, виконання дії, спрацювання тригера), а користувачі є спостерігачами, які отримують відповідні сповіщення.

Під час зміни стану пристрою або виконання сценарію система формує повідомлення та надсилає його користувачу через механізм сповіщень (Notification). Це дозволяє оперативно інформувати користувачів про події в системі без прямої залежності між компонентами.

Таким чином, використання патерну «Спостерігач» забезпечує ефективну взаємодію між компонентами системи, підвищує її модульність та спрощує додавання нових функцій, пов'язаних із обробкою подій.

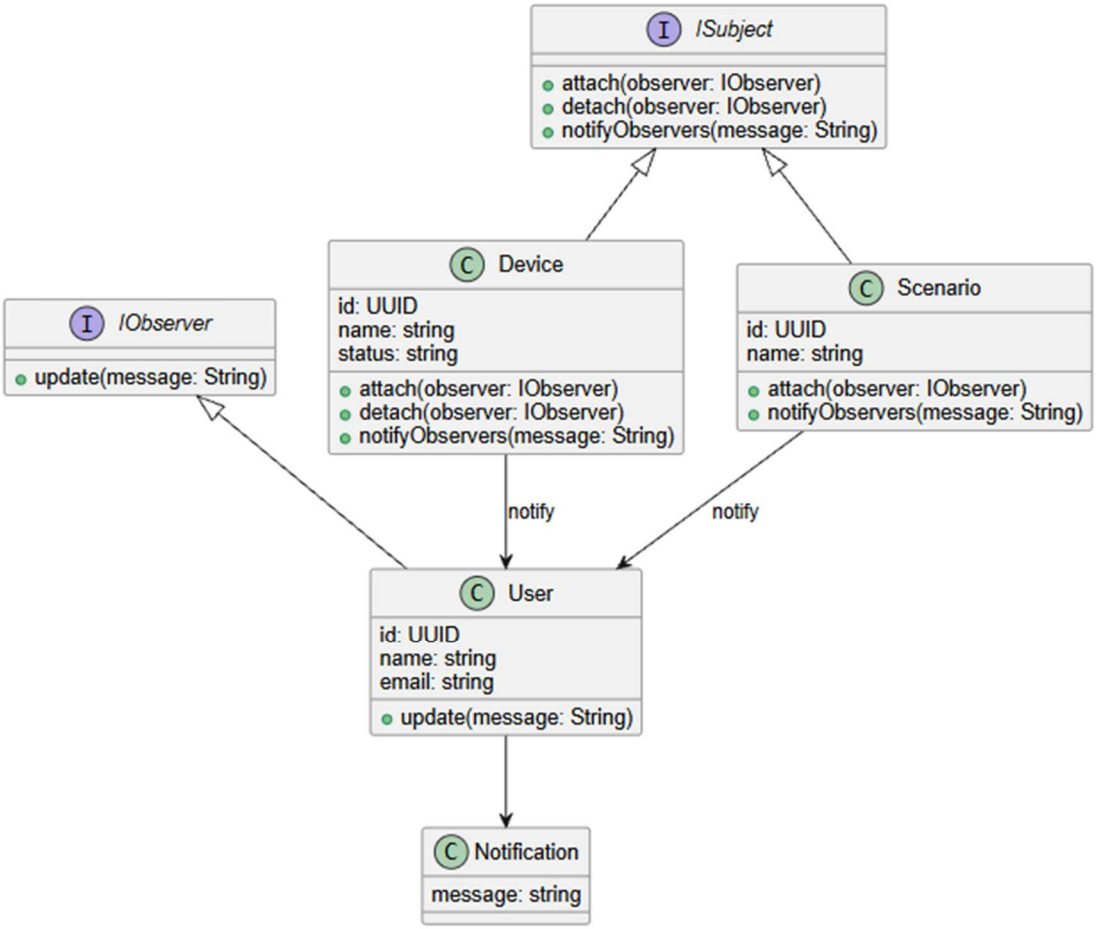

Рис. 2.6. Діаграма класів із застосуванням паттерну «Спостерігач»

На рисунку 2.6 представлено реалізацію патерну «Спостерігач» (Observer), яка базується на використанні двох інтерфейсів (IObserver і ISubject), а також класів Device, Scenario та User. Розглянемо детальніше:

1. Інтерфейс IObserver

Цей інтерфейс визначає поведінку спостерігача:

- Метод ***update(message: String): void*** — реалізує отримання сповіщень про події в системі. Наприклад, користувач може отримувати повідомлення про зміну стану пристрою або виконання сценарію.

2. Інтерфейс ISubject

Інтерфейс визначає механізм роботи суб'єкта:

- ***attach(observer: IObserver): void*** — додає нового спостерігача;
- ***detach(observer: IObserver): void*** — видаляє спостерігача;
- ***notifyObservers(message: String): void*** — надсилає повідомлення всім підписаним спостерігачам.

3. Класи Device / Scenario

Ці класи виступають суб'єктами, які генерують події:

- зберігають список підписаних спостерігачів;
- реалізують методи attach, detach та notifyObservers;
- формують повідомлення при зміні стану пристрою або виконанні сценарію.

4. Клас User

Цей клас реалізує роль спостерігача:

- метод ***update(message: String)*** отримує повідомлення від суб'єкта;
- користувач може бути підписаний на декілька пристроїв або сценаріїв одночасно;
- отримує інформацію про події через механізм сповіщень.

Таким чином, патерн «Спостерігач» забезпечує ефективну взаємодію між компонентами системи, дозволяє відокремити логіку генерації подій від їх обробки та спрощує масштабування платформи.

На рисунку 2.7 представлено реалізацію патерну «Стратегія» (Strategy) у програмній платформі автоматизованого керування житловим середовищем.

Патерн реалізовано через інтерфейс ExecutionStrategy, який визначає загальний метод виконання сценарію. Конкретні реалізації інтерфейсу представлені класами ManualExecution, TimeBasedExecution та SensorBasedExecution, які відповідають за різні способи запуску сценарію.

Клас Scenario виступає контекстом і містить посилання на обрану стратегію виконання. Завдяки цьому система може динамічно змінювати спосіб запуску сценарію без зміни його основної логіки.

Наприклад, сценарій може виконуватись вручну користувачем, автоматично за розкладом або при спрацюванні сенсора. Використання патерну «Стратегія» дозволяє легко додавати нові способи виконання сценаріїв без зміни існуючого коду.

Таким чином, патерн забезпечує гнучкість, розширюваність та зручність підтримки системи.

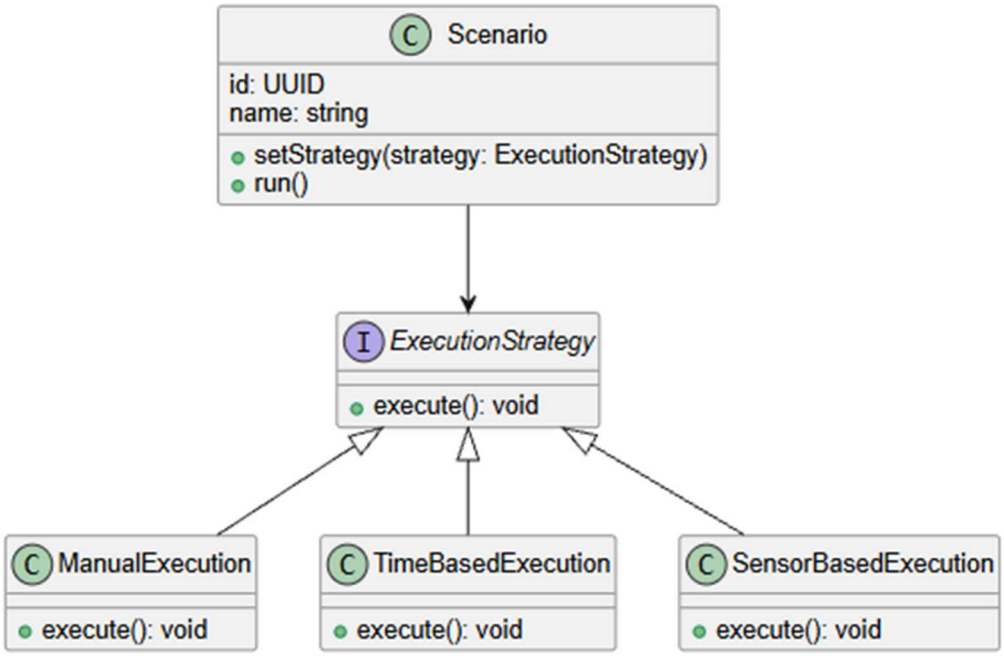

Рис. 2.7. Реалізація патерну «Стратегія» у системі автоматизованого керування житловим середовищем

Реалізований паттерн "Стратегія" для оплати є структурований таким чином:

1. **Інтерфейс ExecutionStrategy**

- визначає метод execute(): void, який реалізується кожною конкретною стратегією виконання;
- інтерфейс задає загальний контракт для різних способів запуску сценарію.

2. **Реалізації стратегії**

ManualExecution

- реалізує виконання сценарію за ініціативою користувача;
- сценарій запускається вручну через інтерфейс системи.

TimeBasedExecution

- реалізує виконання сценарію за розкладом;
- може включати перевірку часу або періодичний запуск.

SensorBasedExecution

- реалізує виконання сценарію на основі даних сенсорів;
- сценарій запускається при досягненні певного значення (наприклад, температура, рух).

3. **Клас Scenario**

- поля:
  - ***executionStrategy: ExecutionStrategy*** — зберігає поточну стратегію виконання сценарію;
- методи:
  - ***run(): void*** — викликає метод ***execute()*** обраної стратегії;
  - ***setStrategy(strategy: ExecutionStrategy)*** — дозволяє змінювати спосіб виконання сценарію під час роботи системи

2.3. Комунікації і послідовність взаємодії об’єктів системи

Діаграми послідовності, кооперації та взаємодії компонентів є важливими засобами для опису функціонування програмної системи. Вони дозволяють наочно відобразити процес обміну даними між об’єктами, послідовність виконання операцій та взаємодію компонентів під час виконання певних задач.

На рисунку 2.9 представлено діаграму послідовності, яка описує процес взаємодії користувача із програмною платформою автоматизованого керування житловим середовищем.

Процес починається з того, що користувач надсилає запит на отримання списку доступних пристроїв. Система обробляє запит, звертається до бази даних і повертає список пристроїв, який відображається користувачу.

Після цього користувач може обрати конкретний пристрій та надіслати команду керування (наприклад, увімкнення або зміна режиму роботи). Система перевіряє права доступу та коректність запиту, після чого передає команду контролеру.

Контролер взаємодіє з фізичним пристроєм, виконує отриману команду та повертає результат виконання до системи. У разі успішного виконання система оновлює стан пристрою в базі даних та надсилає користувачу підтвердження.

Система також може фіксувати виконану дію у журналі подій та, за необхідності, надсилати сповіщення користувачу про зміну стану пристрою.

Таким чином, діаграма послідовності відображає процес взаємодії між користувачем, системою, контролером та пристроєм, демонструючи порядок обміну повідомленнями та виконання операцій.

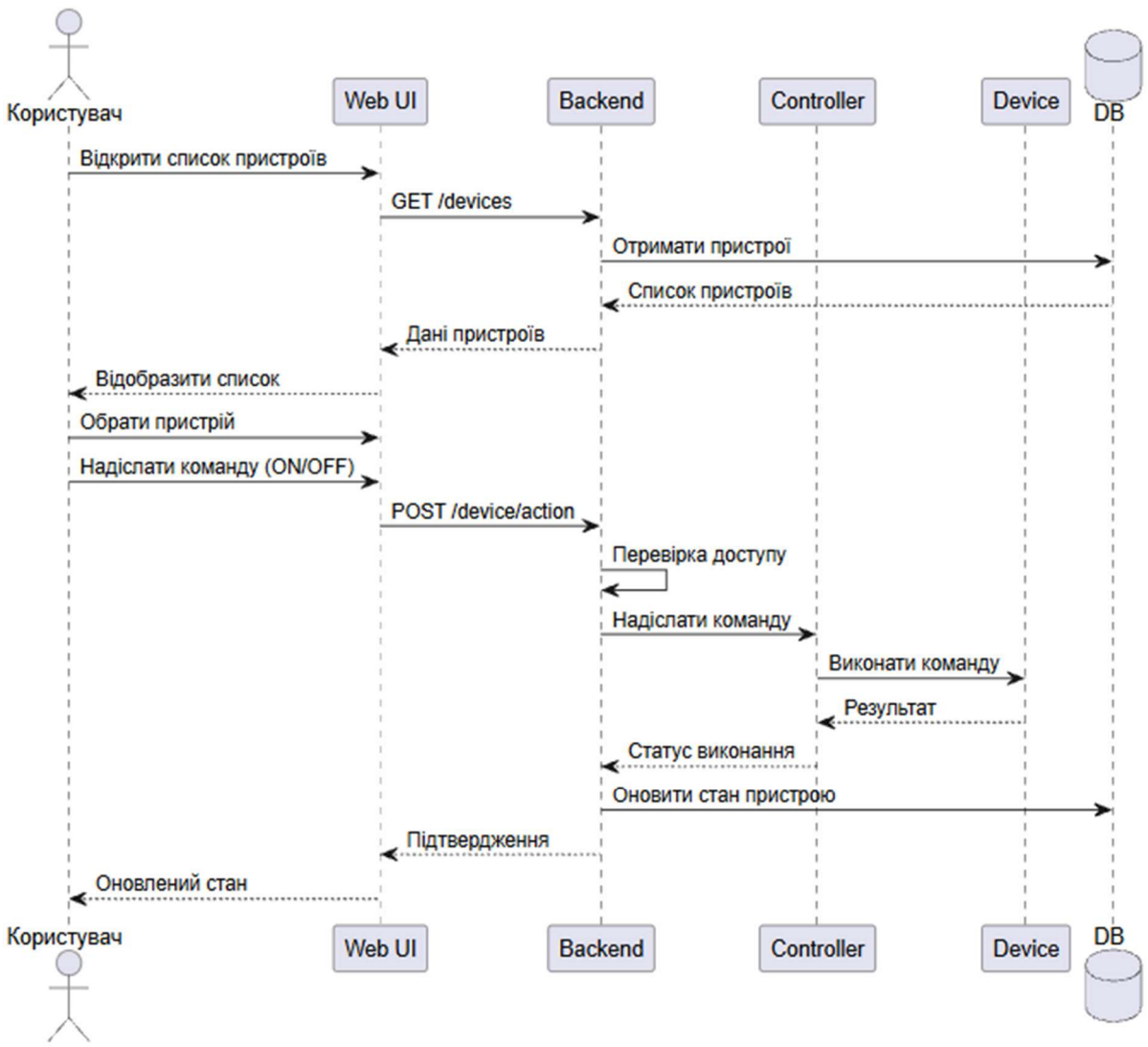

Рис. 2.9. Діаграма послідовності процесу керування пристроєм

Дана діаграма ілюструє чітку послідовність кроків, що забезпечують прозорість і логіку взаємодії.

Діаграма кооперації на рисунку 2.10 відображає взаємодію об’єктів у програмній платформі автоматизованого керування житловим середовищем. Вона описує структуру сценарію, що включає отримання даних про пристрої, обробку запитів користувача, передачу команд до контролера та виконання дій на фізичних пристроях.

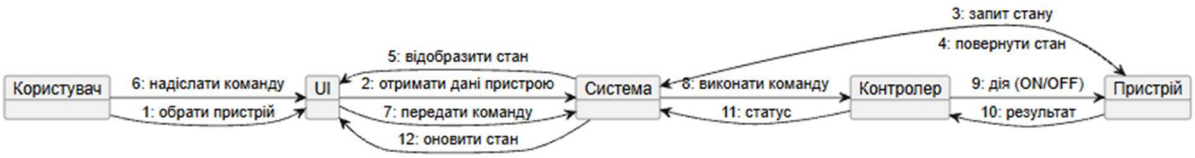

Рис. 2.10. Діаграма кооперації процесу керування пристроєм

Сценарій починається з того, що користувач взаємодіє із системою через інтерфейс для перегляду та керування пристроями. Користувач може отримати список доступних пристроїв, обрати конкретний пристрій та надіслати команду керування.

Запит передається до серверної частини системи, де відбувається перевірка прав доступу та обробка команди. Після цього система передає відповідну команду до контролера, який забезпечує взаємодію з фізичним пристроєм.

Контролер виконує команду на пристрої та повертає результат виконання до системи. У разі успішного виконання система оновлює стан пристрою, зберігає інформацію у журналі подій та відображає актуальні дані користувачу.

Таким чином, система забезпечує чітку взаємодію між користувачем, програмною платформою, контролером та пристроями, що дозволяє ефективно реалізувати процес керування житловим середовищем.

Діаграма компонентів на рисунку 2.11 демонструє архітектуру програмної платформи автоматизованого керування житловим середовищем та взаємодію її основних компонентів.

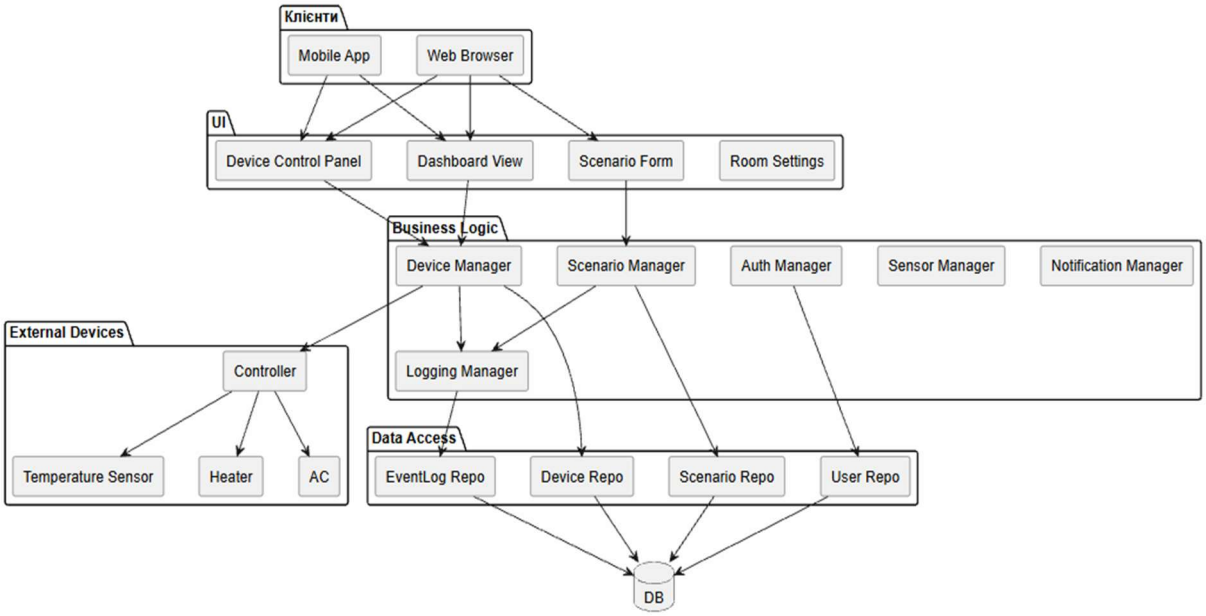

Рис. 2.11. Діаграма компонентів програмної платформи автоматизованого керування житловим середовищем

Головний компонент програмної платформи автоматизованого керування житловим середовищем напряму взаємодіє з ключовими складовими інтерфейсу, такими як DashboardView, DeviceControlPanel, AutomationScenarioForm, RoomSettingsForm та іншими допоміжними компонентами. Кожен із них відповідає за окрему функціональність системи, зокрема перегляд стану пристроїв, керування ними, створення сценаріїв автоматизації та налаштування житлових зон.

Компоненти мають внутрішні залежності між собою. Наприклад, DeviceControlPanel використовує елементи відображення списку пристроїв і компоненти керування їх станом. AutomationScenarioForm містить поля введення параметрів сценарію, вибору тригера та дій. RoomSettingsForm забезпечує роботу з параметрами кімнат і розміщенням пристроїв у межах житлового об’єкта.

Окремі елементи інтерфейсу, такі як кнопки, поля введення, індикатори завантаження та повідомлення, можуть повторно використовуватись у різних частинах системи. Це забезпечує модульність, зменшує дублювання коду та спрощує подальший розвиток програмної платформи.

Таким чином, діаграма компонентів підкреслює чітку організацію складових системи та їхню взаємодію, що забезпечує зручність підтримки, масштабованість і ефективну реалізацію функціональних можливостей платформи.

Окрім цього, діаграма компонентів відображає розподіл відповідальності між окремими частинами системи та їхню взаємодію через чітко визначені інтерфейси. Такий підхід дозволяє ізолювати логіку обробки даних, керування пристроями та взаємодію з користувачем, що підвищує надійність системи та спрощує її масштабування. Завдяки модульній структурі кожен компонент може бути змінений або розширений незалежно від інших, що забезпечує гнучкість архітектури та зручність подальшого розвитку програмної платформи.

Висновки до другого розділу

У другому розділі було розроблено логічну модель програмної платформи автоматизованого керування житловим середовищем. Побудовано діаграми активності та станів, які відображають основні процеси роботи системи, а також діаграму класів, що описує її структуру та взаємозв'язки між основними сутностями.

Крім того, було застосовано патерни проєктування, що забезпечують гнучкість і масштабованість системи, а також побудовано діаграми послідовності та кооперації для відображення взаємодії компонентів.

Отримані результати дозволяють сформувати цілісне уявлення про роботу системи та є основою для її подальшої реалізації і розвитку.

РОЗДІЛ 3. ФІЗИЧНА МОДЕЛЬ ТА ПРОТОТИП ПРОГРАМНОГО КОМПЛЕКСУ

3.1. Взаємодія компонентів системи

У процесі розробки програмної платформи автоматизованого керування житловим середовищем застосовувалися принципи чистої архітектури (рисунок 3.1), які передбачають поділ системи на окремі рівні з чітко визначеними зонами відповідальності. Такий підхід дозволяє відокремити бізнес-логіку від деталей реалізації, зокрема інтерфейсу користувача, бази даних та зовнішніх сервісів.

Основним правилом цієї архітектури є правило залежностей, відповідно до якого залежності в коді повинні бути спрямовані всередину — до більш стабільних, незалежних і абстрактних компонентів. Це забезпечує слабке зв'язування між модулями, підвищує гнучкість системи та спрощує її тестування і подальший розвиток.

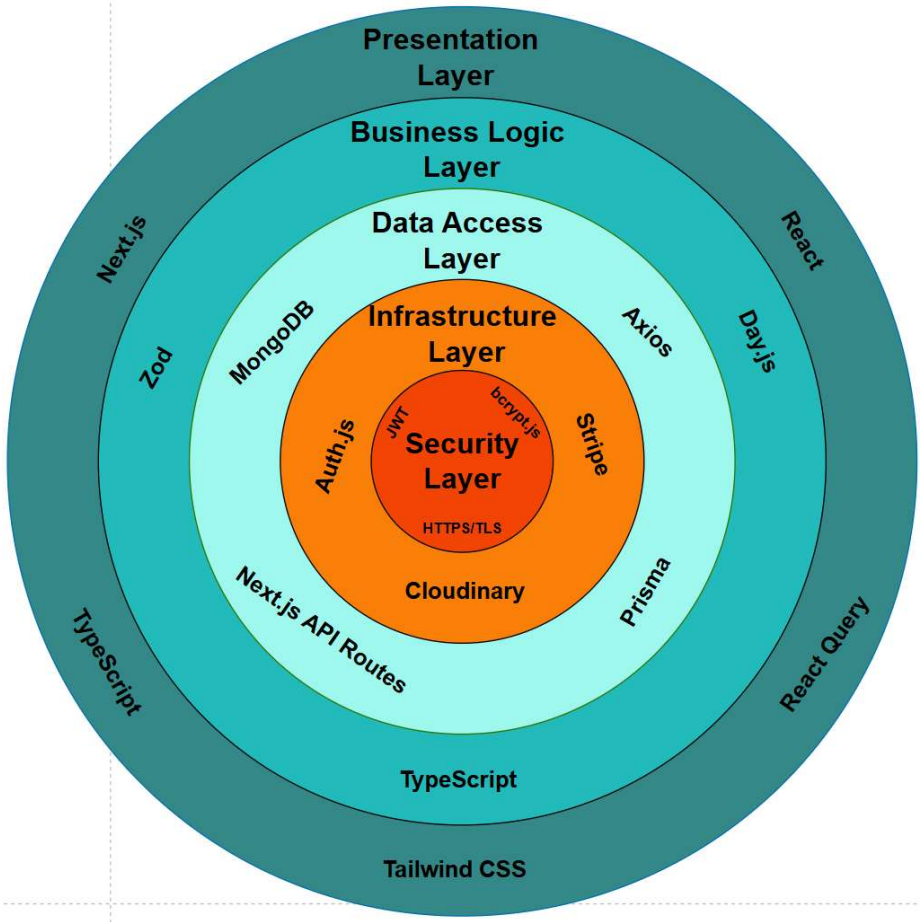

Рис. 3.1. Схема слоїв

Такий підхід сприяє підвищенню модульності та гнучкості програмної платформи автоматизованого керування житловим середовищем: зовнішні компоненти можуть змінюватися або замінюватися без впливу на основну бізнес-логіку. У результаті внутрішні рівні залишаються ізольованими та зосередженими на ключових функціональних завданнях, тоді як зовнішні рівні забезпечують інфраструктуру та інтеграцію із зовнішніми системами.

Нижче наведено опис основних шарів системи, їх призначення та використовуваних технологій:

1. **Security Layer (Шар безпеки)**

Призначення: забезпечує захист даних, автентифікацію та авторизацію користувачів, а також безпечну передачу інформації.

Інструменти:

- JWT — для керування токенами автентифікації;
- bcrypt.js — для хешування паролів;
- HTTPS/TLS — для забезпечення захищеного з'єднання.

2. **Infrastructure Layer (Шар інфраструктури)**

Призначення: забезпечує взаємодію з зовнішніми сервісами та пристроями, зокрема контролерами та IoT-компонентами.

Інструменти:

- Auth.js — для реалізації автентифікації;
- інтеграція з контролерами (IoT Gateway) — для керування фізичними пристроями;
- Axios — для взаємодії із зовнішніми API та сервісами.

3. **Data Access Layer (Шар доступу до даних)**

Призначення: забезпечує збереження, отримання та обробку даних системи.

Інструменти:

- PostgreSQL — для зберігання даних;
- Prisma — ORM для взаємодії з базою даних;
- Next.js API Routes — для реалізації серверної логіки обробки запитів.

4. Business Logic Layer (Шар бізнес-логіки)

Призначення: реалізує основні правила роботи системи, зокрема керування пристроями, виконання сценаріїв та обробку подій.

Інструменти:

- TypeScript — для забезпечення типізації та надійності логіки;
- Day.js — для роботи з датами та часом;
- Zod — для валідації та перевірки даних.

5. Presentation Layer (Шар представлення)

Призначення: забезпечує взаємодію користувача з системою через інтерфейс.

Інструменти:

- React — для створення користувацького інтерфейсу;
- React Query — для управління станом і даними;
- Next.js — для побудови веб-застосунку;
- Tailwind CSS — для стилізації інтерфейсу;
- TypeScript — для типізації компонентів.

3.2. Архітектура програмного комплексу та його розгортання

Діаграма розгортання на рисунку 3.2 використовується для аналізу апаратної структури програмної платформи автоматизованого керування житловим середовищем та визначення конфігурації компонентів, необхідних для її функціонування.

Вузли (англ. node) представляють фізичні або віртуальні компоненти системи, які забезпечують виконання процесів. У даній діаграмі вузлами виступають:

- клієнтські пристрої (Client 1 ... Client N), що представляють користувачів, які взаємодіють із системою через веб-браузер або мобільний застосунок;
- сервер застосунку (Application Server), що реалізує бізнес-логіку та обробку запитів;
- сервер бази даних (Database Server), який забезпечує зберігання та управління даними;
- контролер або IoT-шлюз (Controller), що забезпечує взаємодію системи з фізичними пристроями.

На діаграмі також відображені з’єднання між вузлами:

- HTTP/HTTPS-з’єднання між клієнтами та сервером застосунку для передачі запитів;
- TCP/IP-з’єднання між сервером застосунку та сервером бази даних;
- мережеве з’єднання між сервером застосунку та контролером для передачі команд керування пристроями.

Таким чином, діаграма демонструє структуру розгортання системи та взаємодію її компонентів у реальному середовищі.

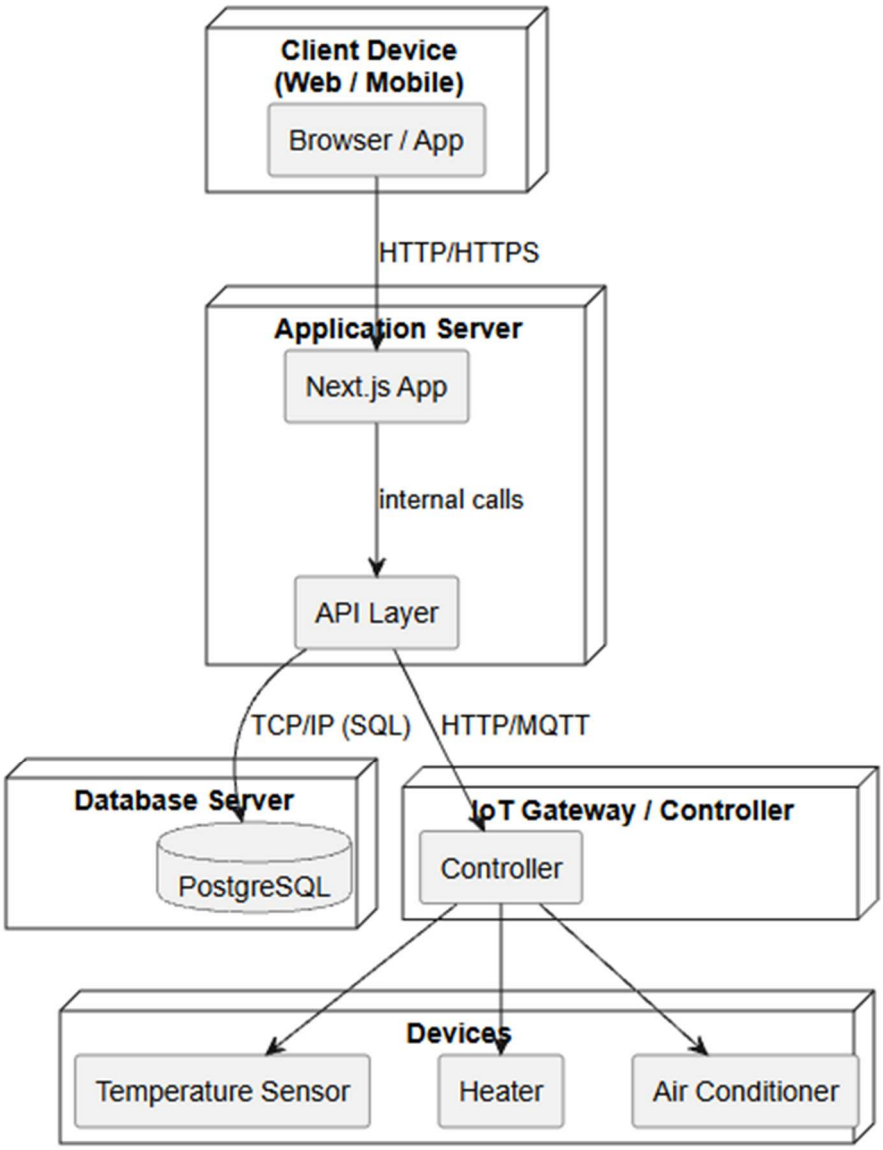

Рис.3.2. Діаграма розгортання програмної платформи автоматизованого керування житловим середовищем

3.3. Генерування програмного коду для прототипу програмного комплексу

Виконання кодогенерації проєкту було здійснено за допомогою середовища розробки Visual Studio 2022, яке надає інструменти для побудови діаграм класів та автоматичного створення програмного коду на їх основі. Такий підхід дозволяє пришвидшити процес розробки та забезпечити відповідність між логічною моделлю системи і її реалізацією.

На основі діаграми класів (рисунок 3.3), створеної в середовищі Visual Studio, було автоматично згенеровано початкову структуру програмного коду. Діаграма класів відображає основні сутності системи автоматизованого керування житловим середовищем, їх атрибути, методи та взаємозв’язки між ними.

У процесі генерації було створено ієрархію класів, що відповідає розробленій моделі, включаючи визначення полів, сигнатур методів і зв’язків між об’єктами. Це забезпечує узгодженість між етапами проєктування та реалізації системи.

Рис. 3.3 – Діаграма класів

Діаграма класів є важливим елементом архітектури системи, оскільки дозволяє наочно представити її структуру. Її використання під час генерації коду спрощує подальшу розробку, тестування та супровід програмного забезпечення, а також полегшує внесення змін і розширення функціоналу.

На рисунку 3.4 наведено перелік файлів, створених у процесі генерації коду. Кожен файл відповідає окремому класу або інтерфейсу, визначеному в діаграмі класів.

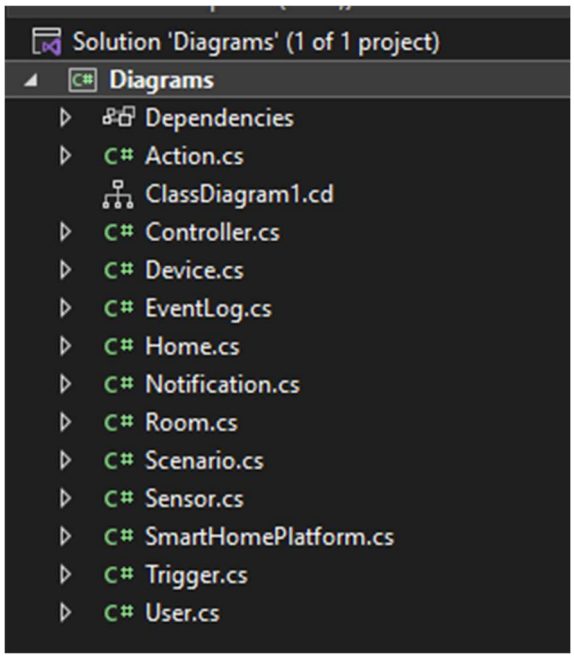

Рис. 3.4 – Список згенерованих файлів

Для демонстрації результатів кодогенерації наведено приклади декількох файлів, створених на основі діаграми класів. Вони містять базову структуру класів, їх атрибути, методи та взаємозв'язки. Повний перелік згенерованих файлів представлено в додатку Б.

Висновки до третього розділу

У третьому розділі було розглянуто архітектуру програмної платформи автоматизованого керування житловим середовищем та особливості її розгортання. Побудовано діаграму розгортання, яка відображає взаємодію між клієнтськими пристроями, сервером застосунку, базою даних та контролерами пристроїв.

Також було реалізовано процес кодогенерації на основі діаграми класів із використанням середовища Visual Studio 2022. У результаті сформовано початкову структуру програмного коду, що відповідає розробленій логічній моделі системи.

Отримані результати підтверджують можливість ефективної реалізації системи, а також забезпечують основу для подальшої розробки, тестування та розширення функціоналу програмної платформи.

ВИСНОВКИ

У ході виконання курсової роботи було проведено аналіз вимог до програмної платформи автоматизованого керування житловим середовищем та сформовано технічне завдання, яке визначає її функціональні та нефункціональні характеристики. Для основних акторів системи було розроблено варіанти використання, що відображають їх взаємодію з програмним комплексом.

Розроблена логічна модель дозволила описати структуру системи, її поведінку та взаємодію компонентів. Для цього були побудовані діаграми активності, станів, класів, послідовності та кооперації. Використання патернів проєктування забезпечило гнучкість архітектури, спростило розширення функціоналу та підвищило рівень модульності системи.

Фізична модель системи відображає основні компоненти розгортання, зокрема клієнтські пристрої, сервер застосунку, базу даних та контролери пристроїв. Такий підхід дозволяє ефективно організувати взаємодію між елементами системи та забезпечити її масштабованість.

У процесі реалізації було виконано кодогенерацію на основі діаграми класів із використанням середовища Visual Studio 2022. Це дозволило сформувати початкову структуру програмного коду, яка відповідає розробленій моделі та може бути використана як основа для подальшої розробки.

Отримані результати підтверджують ефективність обраного підходу до проєктування, забезпечують узгодженість між етапами аналізу, моделювання та реалізації, а також створюють основу для подальшого розвитку програмної платформи.

ДЖЕРЕЛА ІНФОРМАЦІЇ

1. UML Diagram Types Guide [Електронний ресурс]. – 2024. – Режим доступу до ресурсу: https://creately.com/blog/diagrams/uml-diagram-types-examples/.
2. Каграманова Ю. Як будувати UML-діаграми [Електронний ресурс] / Юлія Каграманова. – 2022. – Режим доступу до ресурсу: https://dou.ua/forums/topic/40575/.
3. Діаграма розгортання [Електронний ресурс]. – 2016. – Режим доступу до ресурсу: https://studfile.net/preview/5010027/page:6/.
4. Use Case Diagram - Unified Modeling Language (UML) [Електронний ресурс]. – 2024. – Режим доступу до ресурсу: https://www.geeksforgeeks.org/use-case-diagram/.
5. Observer [Електронний ресурс] // Refactoring.Guru – Режим доступу до ресурсу: https://refactoring.guru/design-patterns/observer.
6. Software Architecture Patterns [Електронний ресурс] // O'Reilly Media – Режим доступу до ресурсу: https://www.oreilly.com/library/view/software-architecture-patterns/9781491971437/ch01.html.
7. Діаграма розгортання [Електронний ресурс] – Режим доступу до ресурсу: https://vuzlit.com/1009782/diagrama_rozgortannya\
8. All You Need to Know about State Diagrams [Електронний ресурс] // Visual Paradigm – Режим доступу до ресурсу: https://www.visual-paradigm.com/guide/uml-unified-modeling-language/about-state-diagrams/.
9. UML Use Case Diagrams [Електронний ресурс] // Kirill Fakhroutdinov – Режим доступу до ресурсу: https://www.uml-diagrams.org/use-case-diagrams.html.
10. Collaboration Diagrams | Unified Modeling Language(UML) [Електронний ресурс]. – 2024. – Режим доступу до ресурсу: https://www.geeksforgeeks.org/collaboration-diagrams-unified-modeling-languageuml/.

11. Zosym M. Діаграма послідовності (Sequence Diagrams) [Електронний ресурс] / Maxym Zosym. – 2022. – Режим доступу до ресурсу: https://www.maxzosim.com/sequence-diagrams/.
12. Моралес Д. Дізнайтеся про важливі деталі діаграми послідовності UML [Електронний ресурс] / Джейд Моралес. – 2023. – Режим доступу до ресурсу: https://www.mindonmap.com/uk/blog/what-is-uml-sequence-diagram/.
13. Петрик М. Р. Моделювання програмного забезпечення [Електронний ресурс] / М. Р. Петрик, О. Ю. Петрик. – 2015. – Режим доступу до ресурсу: https://elartu.tntu.edu.ua/bitstream/123456789/17796/1/%D0%9C%D0%BE%D0%B4%D0%B5%D0%BB%D1%8E%D0%B2%D0%B0%D0%BD%D0%BD%D1%8F%20%D0%BF%D1%80%D0%BE%D0%B3%D1%80%D0%B0%D0%BC%D0%BD%D0%BE%D0%B3%D0%BE%20%D0%B7%D0%B0%D0%B1%D0%B5%D0%B7%D0%BF%D0%B5%D1%87%D0%B5%D0%BD%D0%BD%D1%8F.pdf
14. Діаграма компонентів [Електронний ресурс]. – 2024. – Режим доступу до ресурсу: https://uk.wikipedia.org/wiki/%D0%94%D1%96%D0%B0%D0%B3%D1%80%D0%B0%D0%BC%D0%B0_%D0%BA%D0%BE%D0%BC%D0%BF%D0%BE%D0%BD%D0%B5%D0%BD%D1%82%D1%96%D0%B2.

ДОДАТКИ

Додаток А

Каталог вимог

Бізнес вимоги

1. Основні цілі:

Система розробляється для створення програмної платформи автоматизованого керування житловим середовищем, яка забезпечує централізоване управління пристроями, моніторинг стану та автоматизацію процесів у будинку.

2. Представлення проєкту:

Проєкт буде реалізовано у вигляді веб-застосунку, що надаватиме користувачам інструменти для керування пристроями, створення сценаріїв автоматизації та отримання інформації про стан житлового середовища.

Вимоги користувачів

4. Можливість реєстрації та авторизації користувачів.
5. Можливість перегляду списку пристроїв та їх поточного стану.
6. Можливість керування пристроями (увімкнення, вимкнення, зміна параметрів).
7. Можливість створення, редагування та видалення сценаріїв автоматизації.
8. Можливість налаштування тригерів (час, сенсори, стан пристроїв).
9. Можливість отримання сповіщень про події в системі.
10. Можливість перегляду історії подій та змін стану системи.
11. Можливість групування пристроїв за кімнатами або зонами.
12. Можливість адміністраторам керувати користувачами та налаштуваннями системи.

Функціональні вимоги

Аутентифікація та авторизація користувачів: система повинна забезпечувати реєстрацію, вхід у систему та розмежування доступу за ролями.

Керування пристроями: система повинна дозволяти користувачам надсилати команди пристроям та отримувати їх актуальний стан.

Моніторинг сенсорів: система повинна отримувати та відображати дані з сенсорів у реальному часі.

Створення сценаріїв автоматизації: система повинна дозволяти створювати сценарії, що виконуються автоматично при заданих умовах.

Обробка тригерів: система повинна реагувати на події (час, сенсори, стан пристроїв) та запускати відповідні сценарії.

Журнал подій: система повинна зберігати історію подій і дій користувачів.

Сповіщення: система повинна надсилати повідомлення про важливі події.

Адаптивність: система повинна коректно працювати на різних пристроях (ПК, планшети, смартфони).

Масштабованість: система повинна підтримувати збільшення кількості користувачів і пристроїв.

Нефункціональні вимоги

1. Часова ефективність:
   - час реакції системи — до 1 секунди;
   - виконання команд керування — до 2 секунд;
   - оновлення даних сенсорів — до 1 секунди.
2. Доступність системи:
   - доступність системи — не менше 99%;
   - підключення до бази даних — до 2 секунд.
3. Ресурси:
   - мінімальні вимоги до клієнтського пристрою — 4 Гб оперативної пам’яті;
   - сервер повинен забезпечувати обробку великої кількості запитів без втрати продуктивності.
4. Надійність:
   - система повинна стабільно працювати при одночасному доступі великої кількості користувачів;

- повинна бути передбачена обробка помилок і відновлення після збоїв.

Додаток Б

Глосарій

Програмна платформа автоматизованого керування житловим середовищем (ППАКЖС) — це програмна система, призначена для централізованого керування пристроями житлового середовища, моніторингу їх стану та автоматизації процесів на основі заданих сценаріїв і тригерів.

Додаток В

Лістинг згенерованого файлу Action.cs:

```
using System;
using System.Collections.Generic;

namespace SmartHomePlatform
{
    public class Action
    {
        public Guid Id { get; set; }
        public ActionType Type { get; set; }
        public string ParametersJson { get; set; } = string.Empty;
        public int OrderIndex { get; set; }

        public Guid ScenarioId { get; set; }
        public Scenario? Scenario { get; set; }
    }
}
```

Лістинг згенерованого файлу Controller.cs:

```
using System;
using System.Collections.Generic;

namespace SmartHomePlatform
{
    public class Controller
    {
        public Guid Id { get; set; }
        public string Name { get; set; } = string.Empty;
        public string IpAddress { get; set; } = string.Empty;
        public bool IsOnline { get; set; }

        public Guid HomeId { get; set; }
        public Home? Home { get; set; }

        public List<Device> Devices { get; set; } = new();
    }
}
```

Лістинг згенерованого файлу User.cs:

```
using System.Collections.Generic;

namespace SmartHomePlatform
{
    public class User
    {
        public Guid Id { get; set; }
        public string FullName { get; set; } = string.Empty;
        public string Email { get; set; } = string.Empty;
        public string PasswordHash { get; set; } = string.Empty;
        public UserRole Role { get; set; }
        public bool IsActive { get; set; }
        public DateTime CreatedAt { get; set; }

        public List<Home> Homes { get; set; } = new();
        public List<Notification> Notifications { get; set; } = new();
    }
}
```

Лістинг згенерованого файлу Device.cs:

```
using System;
using System.Collections.Generic;

namespace SmartHomePlatform
{
    public class Device
    {
        public Guid Id { get; set; }
        public string Name { get; set; } = string.Empty;
        public string Type { get; set; } = string.Empty;
        public DeviceStatus Status { get; set; }
        public bool IsEnabled { get; set; }

        public Guid RoomId { get; set; }
        public Room? Room { get; set; }

        public Guid ControllerId { get; set; }
        public Controller? Controller { get; set; }

        public List<Sensor> Sensors { get; set; } = new();
    }
}
```

Лістинг згенерованого файлу EventLog.cs:

```
using System;
using System.Collections.Generic;

namespace SmartHomePlatform
{
    public class EventLog
    {
        public Guid Id { get; set; }
        public string EventType { get; set; } = string.Empty;
        public string Source { get; set; } = string.Empty;
        public string Description { get; set; } = string.Empty;
        public DateTime CreatedAt { get; set; }

        public Guid HomeId { get; set; }
        public Home? Home { get; set; }
    }
}
```

Лістинг згенерованого файлу Home.cs:

```
using System;
using System.Collections.Generic;

namespace SmartHomePlatform
{

    public class Home
    {
        public Guid Id { get; set; }
        public string Name { get; set; } = string.Empty;
        public string Address { get; set; } = string.Empty;
        public DateTime CreatedAt { get; set; }

        public List<Room> Rooms { get; set; } = new();
        public List<Controller> Controllers { get; set; } = new();
        public List<Scenario> Scenarios { get; set; } = new();
        public List<EventLog> EventLogs { get; set; } = new();
    }

}
```

Лістинг згенерованого файлу Notification.cs:

```
using System;
using System.Collections.Generic;

namespace SmartHomePlatform
{
    public class Notification
    {
        public Guid Id { get; set; }
        public NotificationType Type { get; set; }
        public string Message { get; set; } = string.Empty;
        public bool IsRead { get; set; }
        public DateTime CreatedAt { get; set; }

        public Guid UserId { get; set; }
        public User? User { get; set; }
    }
}
```

Лістинг згенерованого файлу Room.cs:

```
using System;
using System.Collections.Generic;

namespace SmartHomePlatform
{
    public class Room
    {
        public Guid Id { get; set; }
        public string Name { get; set; } = string.Empty;
        public int Floor { get; set; }

        public Guid HomeId { get; set; }
        public Home? Home { get; set; }

        public List<Device> Devices { get; set; } = new();
    }
}
```

Лістинг згенерованого файлу Scenario.cs:

```
using System;
using System.Collections.Generic;

namespace SmartHomePlatform
{
    public class Scenario
    {
        public Guid Id { get; set; }
        public string Name { get; set; } = string.Empty;
        public string Description { get; set; } = string.Empty;
        public bool IsActive { get; set; }

        public Guid HomeId { get; set; }
        public Home? Home { get; set; }

        public Trigger? Trigger { get; set; }
        public List<Action> Actions { get; set; } = new();
    }
}
```

Лістинг згенерованого файлу Sensor.cs:

```
using System;
using System.Collections.Generic;

namespace SmartHomePlatform
{
```

```
    public class Sensor
    {
        public Guid Id { get; set; }
        public string Type { get; set; } = string.Empty;
        public string Unit { get; set; } = string.Empty;
        public double Value { get; set; }
        public DateTime UpdatedAt { get; set; }

        public Guid DeviceId { get; set; }
        public Device? Device { get; set; }
    }
}
```

Лістинг згенерованого файлу SmartHomePlatform.cs:

```
using System;
using System.Collections.Generic;

namespace SmartHomePlatform
{
    public enum UserRole
    {
        User,
        Admin
    }

    public enum DeviceStatus
    {
        Off,
        On,
        Standby,
        Error
    }

    public enum TriggerType
    {
        Time,
        Sensor,
        DeviceState
    }

    public enum ActionType
    {
        TurnOn,
        TurnOff,
        ChangeMode,
        SetValue,
        Notify
    }

    public enum NotificationType
    {
        Info,
        Warning,
        Alert
    }
}
```

Лістинг згенерованого файлу Trigger.cs:

```
using System;
using System.Collections.Generic;

namespace SmartHomePlatform
{
    public class Trigger
    {
```

```
        public Guid Id { get; set; }
        public TriggerType Type { get; set; }
        public string ParametersJson { get; set; } = string.Empty;

        public Guid ScenarioId { get; set; }
        public Scenario? Scenario { get; set; }
    }
}
```